Unit 4: Software Coding and Testing

This PPT covers **coding standards, code review, software documentation, software testing strategies, white-box testing, and black-box testing**. Below is a topic-by-topic explanation in simple language, followed by a quick revision section.

1. Coding Standards and Guidelines

Meaning of coding standards

Coding standards are a collection of **rules, guidelines, and best practices** that developers follow while writing programs.

They define how code should be:

- Written.
- Organized.
- Named.
- Indented.
- Commented.
- Tested.
- Protected from errors.

The main goal is to produce code that is **clean, readable, maintainable, reusable, and less error-prone**. [1]

Important coding standards

1. Rules for global variables

Global variables can be accessed from many parts of a program. Excessive use of global variables can create unexpected changes and make debugging difficult.

Therefore, standards specify:

- Which data can be declared globally.
- Which data must remain local.
- When global variables are allowed.

2. Naming conventions

Naming conventions provide a consistent style for variables, functions, classes, and constants.

Example:

```
Global variable: TotalCount
Local variable: totalCount
Constant: MAX_SIZE
Function: calculateTotal()
```

Meaningful names make code easier to understand.

3. Module headers

Every module or file may begin with a standard header containing:

- Module name.
- Author name.
- Date of creation.
- Purpose of the module.
- Modification history.
- Copyright information.

A common header format helps developers understand files quickly.

4. Error-return conventions

An organization should use a consistent method for reporting errors.

For example, a function might:

- Return `0` for failure and `1` for success.
- Return a special error code.
- Throw an exception.
- Return an error object.

The important point is consistency throughout the project.

2. Importance of Coding Standards

Coding standards are useful for the following reasons:

Benefit	Explanation
Consistency	All developers follow a similar coding style.
Readability	Code is easier to read and understand.

Benefit	Explanation
Fewer errors	Common programming mistakes are reduced.
Scalability	The software can be extended more easily.
Team collaboration	Developers can work together smoothly.
Better code reviews	Reviewers can focus on logic rather than formatting.
Easy maintenance	Debugging and updating become simpler.

Example

Without standards:

```
int x;
int total_marks;
int StudentName;
```

With a consistent standard:

```
int totalMarks;
String studentName;
```

The second version is easier to understand.

3. Coding Guidelines

Important coding guidelines mentioned in the PPT are:

Avoid difficult coding styles

Do not write code that is unnecessarily complex or confusing.

Bad practice:

```
result = condition ? a > b ? a : b : 0;
```

A clearer version may use normal `if-else` statements.

Do not use one identifier for multiple purposes

A variable should have one clear purpose.

Bad example:

```
int data;
```

The variable `data` could represent marks, age, or a file size. Use meaningful names instead:

```
int studentMarks;
```

Write well-documented code

Comments should explain:

- Why a particular method is used.
- What a complex algorithm does.
- Important assumptions.
- Any limitations.

Comments should not simply repeat obvious code.

Keep functions short

The PPT recommends that a function should not exceed approximately **10 source lines**. Short functions are:

- Easier to understand.
- Easier to test.
- Easier to reuse.
- Easier to debug.

Avoid `goto`

The `goto` statement can make program flow difficult to follow. Structured programming uses:

- `if-else`.
- `for`.
- `while`.
- `switch`.
- Functions.

4. Coding Best Practices

Use industry-specific standards

Coding requirements may differ according to the industry.

For example:

- Banking software requires strong security and auditability.

- Automobile software may require safety standards.
- Gaming software may focus heavily on performance.
- Medical software may require strict reliability.

Therefore, coding standards should match the type of product being developed.

Focus on readability

Readable code should:

- Use meaningful names.
- Use proper indentation.
- Avoid unnecessarily long lines.
- Avoid deeply nested logic.
- Avoid lengthy functions.
- Use a simple and consistent style.

Meaningful names

Names should clearly express purpose.

Poor names:

```
int x;
int temp;
void process();
```

Better names:

```
int employeeSalary;
int temperature;
void calculateAverage();
```

Avoid deep nesting

Deep nesting means placing many conditions inside one another.

Example of difficult code:

```
if (a) {
    if (b) {
        if (c) {
            if (d) {
                // logic
            }
        }
```

```
        }
    }
```

Deep nesting makes code difficult to test and maintain.

Add comments

Comments are useful for explaining complicated logic, but excessive or outdated comments should be avoided.

Efficient data processing

Large programs should be divided into smaller modules or functions. This supports:

- Reusability.
- Easy testing.
- Easy debugging.
- Easier maintenance.

5. Code Review

Meaning

Code review is a systematic process in which one or more developers examine another developer's code before it is merged into the main codebase.

Code reviews are commonly performed using:

- GitHub.
- GitLab.
- Bitbucket.
- Other version-control platforms. [1]

Purpose of code review

Ensure code quality

Reviewers check whether the code:

- Works correctly.
- Is understandable.
- Follows project standards.
- Is efficient.

Catch bugs early

Errors found during review are cheaper to fix than errors found after deployment.

Promote knowledge sharing

Developers learn:

- New techniques.
- Better coding methods.
- Project architecture.
- Common mistakes.

Enforce coding standards

Reviewers ensure that code follows the team's agreed rules.

Improve collaboration

Code review creates technical discussion and encourages developers to share feedback.

6. Types of Code Review

The PPT discusses two major types:

1. Code walkthrough.
2. Code inspection.

7. Code Walkthrough

A code walkthrough is an **informal code analysis technique**.

Process

1. The developer gives the code to other team members.
2. Team members read and understand the code.
3. Each member selects test cases.
4. They manually simulate the execution of the code.
5. They record their findings.
6. The findings are discussed in a meeting.
7. The original coder is present during the meeting.

Main objective

The main objective is to find:

- Algorithmic errors.
- Logical errors.
- Incorrect calculations.
- Missing conditions.
- Unexpected behavior.

Example

Suppose a program calculates an average. During a walkthrough, a reviewer may manually check:

- What happens when there are no values?
- What happens when all values are zero?
- Is integer division causing an incorrect result?

8. Code Inspection

Code inspection is a more formal review technique.

Main objectives

It identifies:

- Common programming errors.
- Violations of coding standards.
- Incorrect logic.
- Missing error handling.
- Poor documentation.

Inspection team

An inspection team generally consists of **three to eight members**.

Typical roles include:

- **Author:** Person who wrote the code.
- **Reader:** Explains or presents the code.
- **Inspector:** Searches for errors.
- **Recorder:** Records defects and observations.

Difference between walkthrough and inspection

Code Walkthrough	Code Inspection
Informal process.	More formal process.
Focuses mainly on logic and algorithm.	Focuses on common errors and standards.
Team members simulate code manually.	Team members inspect code systematically.
Usually less structured.	Uses defined roles and procedures.

9. Classical Programming Errors

Common programming errors include:

Uninitialized variables

Using a variable before assigning a value.

```
int total;
System.out.println(total);
```

Jumps into loops

Entering the middle of a loop or control structure can cause unexpected behavior.

Nonterminating loops

A loop never ends because its condition never becomes false.

```
while (true) {
    // infinite loop
}
```

Array index out of bounds

Accessing an invalid array position.

```
int[] values = {10, 20, 30};
System.out.println(values[^1_3]); // Error
```

Improper memory allocation or deallocation

Memory may be wasted, leaked, or accessed incorrectly.

Parameter mismatch

The actual arguments passed to a function may not match the expected formal parameters.

Incorrect logical operators

Using `&&` instead of `||`, or using the wrong comparison operator.

Improper loop-variable modification

The loop variable may not be updated correctly, causing an infinite loop or skipped values.

10. Code Review Workflow

A typical code review workflow is:

1. Developer writes the code.
2. Developer submits the code for review.
3. Reviewers examine correctness, style, security, and performance.
4. Reviewers leave comments.
5. Developer updates the code.
6. Reviewers check the updated code.
7. Code is merged into the main branch.

11. Software Documentation

Software development produces several documents, such as:

- User manual.
- Design document.
- Test document.
- Installation manual.
- Software Requirements Specification, or SRS.
- Maintenance documentation.

Software documentation is divided into:

1. Internal documentation.
2. External documentation.

12. Internal Documentation

Internal documentation is documentation provided inside the source code.

It includes:

- Module headers.
- Comments.
- Meaningful variable names.
- Function headers.
- Proper code structure.
- Enumerated types.
- Constants.
- User-defined data types.

Example

```
// Calculates the total marks of a student
int calculateTotalMarks(int[] marks) {
    ...
}
```

Good names are also a form of documentation:

```
int totalMarks;
```

is better than:

```
int x;
```

Internal documentation helps developers understand, debug, and maintain the code.

13. External Documentation

External documentation is provided separately from the source code.

Examples include:

- User manual.
- SRS document.
- Design document.
- Test plan.
- Test report.

- Installation guide.
- Maintenance guide.

External documentation helps different stakeholders understand and use the software.

Internal vs external documentation

Internal Documentation	External Documentation
Exists inside source code.	Exists as separate documents.
Includes comments and names.	Includes manuals, SRS, and test documents.
Mainly helps developers.	Helps users, testers, customers, and developers.

14. Software Testing

Software testing is the systematic process of evaluating software.

It checks whether software:

- Works correctly.
- Meets requirements.
- Is free from defects.
- Handles invalid input.
- Performs acceptably.
- Meets user expectations.

Main goals of testing

- Find bugs.
- Find missing functionality.
- Find errors early.
- Verify requirements.
- Improve software quality.
- Reduce risk after deployment.

15. Verification and Validation

Verification and validation are important concepts in software engineering.

Verification

Verification asks:

> **Are we building the product right?**

It checks whether the software is being developed according to:

- Requirements.
- Design.
- Standards.
- Specifications.

Verification is generally a **static process**, meaning the software does not necessarily need to be executed.

Examples:

- Reviewing requirements.
- Reviewing designs.
- Reviewing source code.
- Checking documents.
- Performing inspections.

Verification is commonly associated with the Quality Assurance team.

Validation

Validation asks:

> **Are we building the right product?**

It checks whether the completed software satisfies the customer's real needs.

Validation is a **dynamic process**, meaning the software is executed and tested.

Examples:

- Running test cases.
- Testing user workflows.
- Performing acceptance testing.
- Checking performance.
- Testing the product in realistic conditions.

Verification vs validation

Verification	Validation
Are we building the product right?	Are we building the right product?
Usually static.	Usually dynamic.
Does not require executing code.	Requires executing software.
Checks documents, design, and code.	Checks actual software behavior.
Often performed by QA.	Often performed by testing team or users.

16. Testing Strategies

The main testing levels in the PPT are:

1. Unit testing.
2. Integration testing.
3. Validation testing.
4. System testing.

17. Unit Testing

A unit is the smallest testable part of a software system.

A unit may be:

- A function.
- A method.
- A class.
- A source-code file.
- A small module.

Unit testing checks each part independently before testing the complete system.

Main focus of unit testing

Unit testing checks:

- Internal processing logic.
- Data structures.
- Input and output flow.
- Local data.
- Independent paths.
- Boundary conditions.

- Error-handling paths.

Important unit-testing activities

Check input and output

Verify that data enters and leaves the module correctly.

Check local data structures

Make sure temporary data is stored and updated correctly.

Test independent paths

All important control-flow paths should be executed at least once.

Test boundary conditions

If valid marks are from 0 to 100, test:

```
-1, 0, 1, 99, 100, 101
```

Test error-handling paths

Check whether the module behaves correctly when errors occur.

18. Drivers and Stubs

Sometimes a module must be tested before the modules connected to it are ready. In this situation, **drivers and stubs** are used.

Suppose the system has:

```
Module A → Module B → Module C
```

If Module B is ready but Modules A and C are not ready:

- A driver can act as Module A.
- A stub can act as Module C.

Driver

A driver is a **dummy main program** used to call and test the module under test.

Characteristics:

- Used mainly in bottom-up testing.

- Simulates a higher-level module.
- Supplies input to a lower-level module.
- Acts as a temporary caller.

Stub

A stub is a **dummy subprogram** that replaces a lower-level module.

Characteristics:

- Used mainly in top-down testing.
- Simulates a lower-level component.
- Returns predefined responses.
- Acts as a temporary called module.

Driver vs stub

Driver	Stub
Dummy main program.	Dummy subprogram.
Calls the module being tested.	Is called by the module being tested.
Simulates higher-level modules.	Simulates lower-level modules.
Common in bottom-up testing.	Common in top-down testing.

19. Integration Testing

Integration testing checks the interfaces and interactions between two or more modules.

The objective is to discover problems such as:

- Incorrect data passing.
- Interface mismatch.
- Wrong sequence of calls.
- Communication errors.
- Incompatible assumptions.

There are three major integration approaches.

20. Big Bang Integration Testing

In the Big Bang approach:

1. All modules are developed and unit-tested.
2. All modules are combined at once.

3. The complete integrated system is tested.

Advantage

- Simple to understand.
- Does not require a detailed integration sequence.

Disadvantage

- Difficult to identify the source of errors.
- Many modules are combined simultaneously.
- Debugging becomes difficult.

21. Bottom-Up Integration Testing

Testing begins with the lowest-level modules and moves upward.

Process

1. Test low-level modules first.
2. Combine them into larger modules.
3. Continue toward higher-level modules.
4. Integrate the complete system.

Drivers may be used when higher-level modules are not ready.

Advantage

- Low-level utility modules are tested early.
- Errors in basic modules are discovered early.

Disadvantage

- The complete user-level behavior is tested late.
- Drivers may need to be created.

22. Top-Down Integration Testing

Testing begins with the highest-level modules and moves downward.

Process

1. Test the main or top-level module.
2. Integrate lower-level modules gradually.
3. Replace unavailable lower-level modules with stubs.
4. Continue until all modules are integrated.

Advantage

- Main system behavior is tested early.
- The overall system structure can be demonstrated quickly.

Disadvantage

- Stubs must be developed.
- Low-level modules may be tested late.

Integration approaches

Approach	Starting point	Temporary component
Big Bang	All modules together	Usually none
Bottom-up	Lowest-level modules	Drivers
Top-down	Highest-level modules	Stubs

23. Regression Testing

Regression testing means repeatedly testing an already-tested program after modifications.

The purpose is to discover whether new changes have introduced defects into previously working functionality.

Example

A website already supports email login. Facebook login is added. Regression testing checks:

- Whether Facebook login works.
- Whether email login still works.
- Whether logout still works.
- Whether password recovery is affected.

When is regression testing performed?

Regression testing is performed after:

- Adding new functionality.
- Changing requirements.
- Fixing a defect.
- Improving performance.
- Changing the environment.
- Updating the database.
- Modifying existing code.

Example situations

Change	Regression test
Add Facebook login	Recheck email login and logout.
Remove forgot-password feature	Check all login functionality.
Fix login button	Recheck the complete login flow.
Improve page loading	Check functionality and performance.
Change MySQL to Oracle	Test all database-related features.

Regression testing gives developers confidence that changes have not caused unexpected side effects.

24. Smoke Testing

Smoke testing is a quick, broad test performed after a new build is released.

Its main purpose is to check **build stability**.

Characteristics

- Performed after each build.
- Usually performed by testers or developers.
- Checks major and critical functionality.
- Does not test every feature in depth.
- Can be used during integration, system, and acceptance testing.

Example smoke tests for an e-commerce website

- Website opens.
- User can log in.
- Product search works.
- Product can be added to cart.
- Checkout page opens.
- Payment page is available.

If a critical feature fails, the build may be rejected and returned to the development team.

Smoke testing vs regression testing

Smoke Testing	Regression Testing
Quick and shallow.	Detailed and broad.
Checks whether the build is stable.	Checks whether changes broke existing features.
Tests critical functions only.	May test many existing functions.
Performed after a new build.	Performed after modifications or fixes.

25. Validation Testing

Validation testing determines whether software satisfies specified business and customer requirements.

It provides assurance that the software meets:

- Informational requirements.
- Functional requirements.
- Behavioral requirements.
- Performance requirements.

For custom software, validation may include acceptance tests performed with the customer or end users.

For commercial software used by many customers, formal testing with every customer is impractical. Therefore, alpha and beta testing are commonly used.

26. Alpha Testing

Alpha testing is performed:

- At the developer’s site.
- By a representative group of end users.

- In a controlled environment.
- With developers observing and recording problems.

The developers may watch users interact with the software and identify:

- Errors.
- Confusing screens.
- Usability problems.
- Unexpected behavior.

Example

A company invites selected users to its office and asks them to use a new application while developers observe them.

27. Beta Testing

Beta testing is performed:

- At one or more end-user locations.
- In a real or live environment.
- Without developers usually being present.
- Under conditions that developers cannot fully control.

Users record problems and report them to developers.

After issues are corrected, the software may be released to the entire customer base.

Alpha vs beta testing

Alpha Testing	Beta Testing
Conducted at developer's site.	Conducted at customer or user sites.
Controlled environment.	Real-world environment.
Developers are present.	Developers are generally absent.
Performed before beta testing.	Performed closer to final release.

28. Acceptance Testing

Acceptance testing is performed to determine whether the customer should accept the system.

It checks compliance with:

- User needs.
- Business requirements.

- Acceptance criteria.
- Business processes.

Acceptance testing is usually performed:

1. After system testing.
2. Before the system is made available for actual use.

The result helps the customer decide whether the software is acceptable.

29. System Testing

System testing tests the complete software and other related system elements together.

These elements may include:

- Hardware.
- Software.
- Database.
- Network.
- People.
- External services.

The objective is to verify that all elements work together and that the complete system performs as expected.

System testing is a collection of different tests.

30. Types of System Testing

The PPT covers:

1. Recovery testing.
2. Security testing.
3. Stress testing.
4. Performance testing.
5. Deployment testing.

31. Recovery Testing

Recovery testing checks whether software can recover after failures.

The tester deliberately causes failures and observes whether recovery is successful.

Examples:

- Power failure.
- Network failure.
- Server crash.
- Database failure.
- Hardware failure.

Automatic recovery

The system may perform:

- Re-initialization.
- Checkpoint recovery.
- Data restoration.
- Restart.

Manual recovery

If human intervention is needed, the mean time to repair, or **MTTR**, is measured.

MTTR means the average time required to restore the system after failure.

32. Security Testing

Security testing checks whether the software protects itself from unauthorized access or attacks.

The tester may act like an attacker and try to:

- Access restricted pages.
- Guess passwords.
- Bypass authentication.
- View another user's data.
- Submit malicious input.
- Access administrative functions.

Security testing verifies the effectiveness of protection mechanisms.

33. Stress Testing

Stress testing runs a system under abnormally high conditions.

It may increase:

- Number of users.

- Number of requests.
- Data volume.
- Processing frequency.
- Memory usage.
- Network traffic.

The purpose is to determine:

- When the system slows down.
- When it fails.
- Whether it fails safely.
- Whether it recovers after overload.

Example

Testing a website with 100,000 users at the same time when the normal load is only 10,000 users.

34. Performance Testing

Performance testing measures the runtime behavior of software.

It checks:

- Response time.
- Processing speed.
- Throughput.
- Resource usage.
- Scalability.
- Stability.

Performance testing can occur at every testing level, including unit testing.

Example

A performance test may check whether a search page returns results within two seconds.

35. Deployment Testing

Deployment testing checks whether software works correctly in every environment where it will operate.

It examines:

- Installation procedures.

- Installation software.
- Configuration steps.
- Operating-system compatibility.
- Database setup.
- Network setup.
- User documentation.

The purpose is to make sure customers can install and start using the software successfully.

36. Black-Box and White-Box Testing

Software can be tested from two views:

1. External view: black-box testing.
2. Internal view: white-box testing.

37. White-Box Testing

White-box testing is also called:

- Glass-box testing.
- Structural testing.
- Clear-box testing.

It examines the internal structure, logic, and implementation of the software.

The tester generally needs knowledge of:

- Source code.
- Control flow.
- Conditions.
- Loops.
- Data structures.
- Internal design.

Main objectives

White-box testing checks that:

- Every independent path is executed.
- Every statement runs at least once.
- Every decision has true and false outcomes.
- Loops work at boundaries and normal values.

- Internal data structures are correct.

Example

For the following condition:

```
if (age >= 18) {
    allowVoting();
} else {
    rejectVoting();
}
```

White-box testing should test:

- `age = 18`, where the condition is true.
- `age = 17`, where the condition is false.

38. Basis Path Testing

Basis path testing is a white-box technique used to measure the logical complexity of a program.

It creates test cases that execute independent logical paths through the program.

Its goal is to ensure that every statement and important control path is executed.

39. Advantages of White-Box Testing

Complete code coverage

It tests:

- Statements.
- Branches.
- Conditions.
- Paths.

Early bug detection

Errors can be found before the user interface or complete system is ready.

Code optimization

It helps identify:

- Unnecessary code.
- Duplicate code.
- Inefficient logic.
- Performance problems.

Improved code quality

It encourages developers to write structured and correct code.

Automation potential

Many white-box tests can be automated using testing tools and frameworks.

Security benefits

It can identify vulnerabilities hidden inside the implementation.

40. Disadvantages of White-Box Testing

Time-consuming

Testing every path can require a lot of time.

Requires expertise

Testers need programming and code-analysis knowledge.

Sensitive to code changes

When the implementation changes, existing test cases may become invalid.

Costly

It may require skilled testers and specialized tools.

May miss functionality problems

White-box testing may confirm that the code works internally but miss:

- Incorrect business requirements.
- Poor user experience.
- Confusing interface behavior.
- Missing functionality.

41. Prime Number Example

The PPT gives a Java method for checking whether a number is prime:

```
public static boolean isPrime(int num) {
    if (num <= 1)
        return false;

    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0)
            return false;
    }

    return true;
}
```

White-box test cases

Prime values:

```
isPrime(2)  → true
isPrime(3)  → true
isPrime(13) → true
```

Non-prime values:

```
isPrime(1)  → false
isPrime(0)  → false
isPrime(-5) → false
```

These tests check different internal paths:

- Input less than or equal to 1.
- Loop execution.
- Divisible number.
- Number with no divisor.
- Final return statement.

42. Black-Box Testing

Black-box testing is also called:

- Behavioral testing.
- Functional testing.

It tests software from the user's or external point of view without considering the internal source code.

The tester provides input and checks the output.

Main focus

Black-box testing focuses on functional requirements.

It checks whether the software:

- Performs the required functions.
- Produces correct outputs.
- Handles invalid input.
- Responds correctly to user actions.
- Meets performance expectations.

Errors found by black-box testing

Black-box testing may discover:

- Missing functions.
- Incorrect functions.
- Interface errors.
- Database-access errors.
- Data-structure errors visible through output.
- Performance errors.
- Initialization errors.
- Termination errors.

43. Questions Answered by Black-Box Testing

Black-box testing asks:

- Does the system behave correctly for valid input?
- Does it handle invalid input properly?
- Are outputs correct for different inputs?
- Does the user interface respond correctly?
- Does the software meet performance requirements?
- Does it work under normal, peak, and stress conditions?

Functional validity

The tester compares:

```
Actual output vs expected output
```

For example, for a login function:

- Correct username and password should allow access.
- Wrong password should reject access.
- Empty fields should display validation messages.
- Locked accounts should be denied.

44. Input Classes in Black-Box Testing

Valid input classes

Inputs that should be accepted.

Example: age from 18 to 60.

Invalid input classes

Inputs that should be rejected.

Example:

- Negative age.
- Text instead of a number.
- Empty input.

Boundary values

Values at the edges of valid ranges.

If the valid range is 1 to 100, test:

```
0, 1, 2, 99, 100, 101
```

This helps find off-by-one errors.

Sensitive input values

Some inputs may cause:

- Crashes.
- Incorrect results.
- Rounding errors.
- Data overflow.
- Validation failures.
- Special business rules.

45. Data Volume and Rate Testing

Black-box testing can also check the amount and speed of data the system can handle.

Load testing

Gradually increases the load to find when the system slows down or fails.

Examples:

- Increasing the number of users.
- Uploading increasingly large files.
- Sending more requests per second.

Stress testing

Pushes the system beyond its expected capacity.

It checks whether the system:

- Fails safely.
- Recovers correctly.
- Gives useful error messages.
- Avoids data corruption.

46. White-Box vs Black-Box Testing

White-Box Testing	Black-Box Testing
Tests internal code structure.	Tests external behavior.
Requires source-code knowledge.	Does not require source-code knowledge.
Focuses on logic, paths, branches, and loops.	Focuses on requirements and functionality.
Usually performed by developers or technical testers.	Can be performed by testers and users.

White-Box Testing	Black-Box Testing
Finds coding and logic errors.	Finds missing functions and interface errors.
Includes basis path testing.	Includes input and boundary testing.
May miss user-experience problems.	May miss internal unexecuted paths.

Quick Revision

Coding Standards

- Coding standards are rules for writing consistent and maintainable code.
- They cover naming, indentation, comments, error handling, and code organization.
- Use meaningful names.
- Avoid global variables when possible.
- Avoid `goto`.
- Keep functions short.
- Avoid deep nesting.
- Divide code into small modules.

Code Review

- Code review means examining code before merging it.
- Main goals: find bugs, ensure quality, share knowledge, and enforce standards.
- Walkthrough is informal.
- Inspection is formal and uses roles such as author, reader, inspector, and recorder.
- Common errors include uninitialized variables, infinite loops, invalid array indexes, and parameter mismatches.

Documentation

- Internal documentation exists inside source code.
- Examples: comments, headers, meaningful names, and proper code structure.
- External documentation exists separately.
- Examples: SRS, design document, user manual, installation manual, and test documents.

Testing Basics

- Testing evaluates software to find defects and verify requirements.
- Verification asks: **Are we building the product right?**
- Validation asks: **Are we building the right product?**
- Verification is mainly static.

- Validation is mainly dynamic.

Testing Levels

- **Unit testing:** Tests the smallest software component.
- **Integration testing:** Tests communication between modules.
- **System testing:** Tests the complete system.
- **Validation testing:** Checks whether software meets customer requirements.
- **Acceptance testing:** Determines whether the customer accepts the system.

Drivers and Stubs

- **Driver:** Dummy main program; commonly used in bottom-up testing.
- **Stub:** Dummy subprogram; commonly used in top-down testing.

Integration Approaches

- **Big Bang:** Combine all modules at once.
- **Bottom-up:** Test low-level modules first; use drivers.
- **Top-down:** Test high-level modules first; use stubs.

Other Testing Types

- **Regression testing:** Re-test after changes.
- **Smoke testing:** Quickly check critical functions after a build.
- **Alpha testing:** At developer's site in a controlled environment.
- **Beta testing:** At user sites in a real environment.
- **Recovery testing:** Checks recovery after failure.
- **Security testing:** Checks protection from unauthorized access.
- **Stress testing:** Tests beyond normal capacity.
- **Performance testing:** Measures speed, response time, and resource use.
- **Deployment testing:** Checks installation and operation in target environments.

White-Box Testing

- Also called structural or glass-box testing.
- Examines internal code logic.
- Tests paths, branches, loops, statements, and data structures.
- Basis path testing measures logical complexity.
- Advantage: high code coverage.
- Disadvantage: requires programming expertise and can be expensive.

Black-Box Testing

- Also called functional or behavioral testing.
- Tests software without examining internal code.
- Checks input, output, functionality, interface, and performance.
- Tests valid inputs, invalid inputs, and boundary values.
- Example for range 1–100: test `0, 1, 2, 99, 100, 101`.

Most Important Differences

- Verification: product is built correctly.
- Validation: correct product is built.
- Walkthrough: informal review.
- Inspection: formal review.
- Driver: simulates caller.
- Stub: simulates called module.
- Smoke testing: checks build stability.
- Regression testing: checks whether changes broke old features.
- White-box: internal structure.
- Black-box: external behavior.

⁂

1. unit-4.pptx